

AeroMoE: A Resource-Efficient Collaborative Aerial Edge AI System for LLM Inference

Abstract—Mixture-of-Experts (MoE) models enable efficient inference through sparse expert activation. They are attractive for onboard intelligence in unmanned aerial vehicle (UAV) edge networks, where low-latency processing over wide areas is required without reliable cloud connectivity. However, deploying MoE models remains difficult. UAV edge networks feature heterogeneous onboard resources, limited link bandwidth, and spatially heterogeneous task demands. Directly loading the full expert set readily exhausts the limited memory of individual UAVs. To address this issue, we propose AeroMoE, a similarity-aware framework for collaborative MoE inference. The framework constructs substitutable expert sets based on similarity and extends basic demands into effective demands. It then computes multi-dimensional expert placement scores considering demand, UAV connectivity, communication rate, computing power, and redundancy cost. A two-stage dynamic greedy algorithm decides expert placements on UAVs to ensure critical coverage while balancing gain and redundancy under memory constraints. Finally, a lightweight per-layer scheduler selects the best local or single-hop UAV to execute each expert step. Simulations show that AeroMoE reduces average end-to-end inference latency by 24.1%, 21.1%, and 48.3% compared with random placement, importance-based placement, and similarity-agnostic deployment, respectively. These results confirm the effectiveness of similarity-aware substitution and communication-aware placement. AeroMoE offers a practical solution for collaborative intelligence in resource-limited aerial edge networks.

Index Terms—UAV edge computing, Mixture-of-Experts, expert deployment, similarity-aware substitution, collaborative inference.

I. INTRODUCTION

With the development of edge intelligence, unmanned aerial vehicle (UAV) edge networks have emerged as a key platform for low-latency task processing [1]. UAVs can be flexibly deployed according to task regions and provide computing and communication services to ground users via wireless communications, making them suitable for emergency sensing, aerial inspection, target recognition, and collaborative monitoring [2]. At the same time, mixture-of-experts (MoE) large language models (LLMs) enable efficient inference by sparsely activating only a subset of experts [3]. This approach expands model capacity while reducing per-inference computation cost, offering a promising solution for intelligent inference in resource-constrained edge environments [4].

However, enabling MoE inference in UAV edge networks is challenging. First, UAVs have limited onboard computing power and memory, making it difficult to deploy the full set of experts on every UAV, and memory constraints become the primary bottleneck as the number or size of experts increases [5], [6]. Second, task demands exhibit strong spatial heterogeneity across UAV service regions, so globally static deployment cannot adapt to localized demand patterns [7]. Third, wireless links between UAVs have limited bandwidth,

and the collaborative inference considered in this paper restricts expert execution to locally available or reachable UAVs [8], [9]. When expert deployment locations do not match task access locations, intermediate features must be frequently transmitted across UAVs, significantly increasing end-to-end latency.

Recent studies have shown that MoE experts contain redundant knowledge and exhibit functional similarity, and have explored clustering or replacement techniques for model compression [10]–[12]. Exchangeability of expert activations has also been observed in resource-constrained edge scenarios [13]. However, these works primarily target model pruning or deployment in cloud and fixed-edge environments. None has formulated similarity-based expert substitution as a deployment and scheduling strategy for UAV edge networks under strict memory constraints. Consequently, existing methods struggle to simultaneously achieve expert coverage and low-latency collaborative inference on memory-limited UAVs.

To address this gap, we propose the AeroMoE framework that minimizes end-to-end inference latency for collaborative MoE inference in UAV edge networks through expert deployment and scheduling under strict memory constraints. AeroMoE first collects basic expert demands per UAV region based on task access locations and constructs substitutable expert sets using expert similarity. It then extends raw demands into effective demands for candidate experts. Next, AeroMoE computes communication-aware placement scores that jointly consider effective demand, UAV connectivity, link rates, computing power, and redundancy cost. A two-stage dynamic greedy algorithm is employed, i.e., a pre-deployment phase ensures coverage of critical experts, followed by a dynamic adjustment phase that selects higher-gain, lower-redundancy placements under remaining memory budgets. Finally, based on the deployment result, AeroMoE performs lightweight per-layer inference scheduling, choosing for each expert step a locally available or single-hop reachable UAV. Extensive simulations in random hotspot UAV scenarios show that AeroMoE reduces average end-to-end inference latency by approximately 24.1%, 21.1%, and 48.3% compared with random placement, importance-based placement, and similarity-agnostic deployment, respectively. These results validate the effectiveness of the proposed framework.

The main contributions of this paper are summarized as follows.

- We develop a system model for collaborative MoE inference in UAV edge networks. The model jointly incorporates single-hop communication links, heterogeneous UAV resources, and per-task expert execution sequences into the end-to-end latency analysis.
- We propose AeroMoE, a similarity-aware expert de-

ployment and scheduling framework for collaborative MoE inference in UAV edge networks. The framework constructs substitutable expert sets via similarity, extends basic demands into effective demands, and performs communication-aware placement under strict memory constraints.

- We design a communication-aware placement scoring mechanism, a two-stage dynamic greedy deployment algorithm, and a lightweight per-layer inference scheduler. Simulation results demonstrate that the proposed methods achieve latency reduction over multiple baselines in UAV edge networks.

The remainder of this paper is organized as follows. Sec. II reviews the background and motivation. Sec. III develops the system model for MoE inference in UAV edge networks. Sec. IV presents the AeroMoE framework. Sec. V provides simulation results, and Sec. VI concludes the paper.

II. BACKGROUND AND MOTIVATION

A. MoE Expert Inference

MoE models expand capacity through multiple expert networks while sparsely activating only relevant experts via a gating network [14], [15]. Unlike dense models, MoE executes only a small subset of experts per input, enabling large parameter counts with controlled computing cost [16]. In task-level inference, a task is represented as an ordered sequence of expert executions. Execution can occur locally if the required expert is available on the current or a single-hop reachable UAV. Otherwise, intermediate features must be transmitted across UAVs [17]. Thus, MoE inference performance in edge networks depends on both computing and placement-induced communication cost. Although sparse activation reduces computation, deployment remains challenging. For memory-constrained UAVs, the core problem is no longer whether to run the full model, but which experts to place on which UAVs.

B. Expert Deployment Challenges in UAV Edge Networks

UAV edge networks impose stricter constraints than fixed-edge servers. Each UAV has heterogeneous memory and computing power [18], [19]. Deploying many or large experts quickly exhausts memory, leaving some required experts unavailable. Task demands also exhibit strong spatial heterogeneity. Different regions generate distinct requests, producing non-uniform expert demand across UAVs [20]. Uniform deployment either wastes memory or fails to cover localized high-demand areas. Communication further complicates placement. UAV-UAV links have limited bandwidth and location-dependent connectivity [21]. When consecutive experts reside on different UAVs, intermediate features must traverse these bandwidth-limited links, incurring significant transmission cost [22]. Expert placement must therefore jointly account for task demand, UAV resources, and communication topology.

C. Related Work

UAV networks provide flexible aerial edge services through on-demand deployment and line-of-sight links, and have been extensively studied for coverage, emergency response, and edge computing [23], [24]. Prior UAV edge computing research mainly focuses on trajectory optimization, task offloading, and resource allocation to reduce latency under communication and energy constraints. However, these works typically treat inference tasks as monolithic loads or consider only conventional DNN offloading, rarely leveraging the sparse activation structure of MoE models. This paper instead addresses collaborative MoE inference at the expert level, while accounting for expert demands, memory limits, and transmission overhead.

Edge intelligence has shown that moving deep inference to the edge reduces cloud dependency and improves real-time performance [25]. Prior collaborative inference systems enhance performance via model partitioning or heterogeneous processor cooperation, e.g., CoEdge [26], CoDL [27], BAND [28], Asymo [29]. These works mainly target conventional DNNs with deployment granularity at the layer or operator level. In contrast, AeroMoE operates at the MoE expert level, and it decides which experts (or similar substitutes) to cache on which UAVs and selects per-step execution locations accordingly.

MoE models achieve large capacity with low per-inference cost via sparse activation, as demonstrated by Sparsely-Gated MoE [30], Switch Transformer [31], GShard [32], and Mixtral [33]. Routing and load-balancing improvements have been explored in Expert-Choice routing [34] and GLaM [35]. System-level MoE serving frameworks, such as DeepSpeed-Inference [36], Orca [37], and AlpaServe [38], mainly target resource-rich cloud and data center environments, emphasizing throughput and parallel execution. In contrast, UAV edge networks feature device heterogeneity, constrained bandwidth, and dynamic single-hop connectivity, rendering data-center-oriented expert-parallel and full-deployment strategies inapplicable.

D. Motivation for Similarity-Aware Deployment

While existing work has studied UAV edge computing, collaborative inference, and MoE serving optimization, none has explored similarity-based expert substitution in memory-constrained UAV edge networks. This creates a specific deployment dilemma. Standard deployment assumes each requested expert must execute itself, so deploying all original experts exceeds memory limits, while deploying only frequent experts leaves necessary low-frequency demands unserved. Expert similarity offers a solution. If an expert is sufficiently similar to a required expert, it can serve as a substitute, allowing fewer deployed experts to cover multiple demands [39]. However, substitution introduces new challenges. Approximation error may accumulate over a task sequence, while deploying similar experts nearby causes redundancy. Besides, the benefit of substitution depends heavily on the communication cost. A high-similarity substitute on a poorly connected UAV may still incur high latency. Similarity-aware deployment

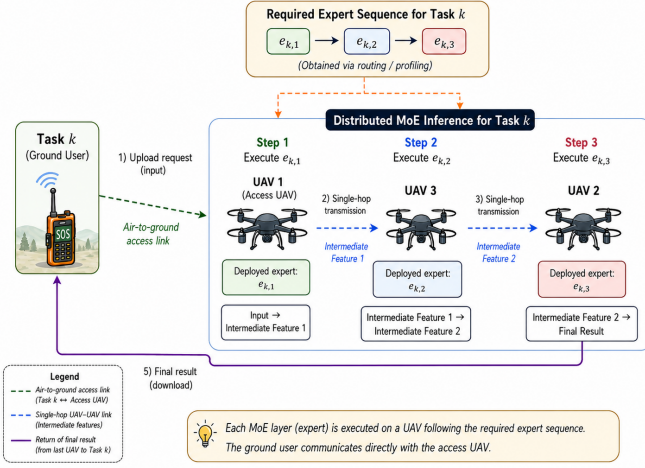


Fig. 1. Collaborative MoE inference scenario in a quasi-static UAV edge network. Ground tasks access UAVs through air-to-ground links. Heterogeneous UAVs jointly execute MoE inference units over a given wireless topology, with intermediate features transmitted via UAV-UAV links. Final results are returned to a base station or emergency command center.

must therefore jointly optimize candidate selection, effective demand computation, placement, and redundancy control under memory and communication constraints. These issues motivate the AeroMoE framework.

III. SYSTEM MODEL

To focus on the LLM inference, we assume that the positions of UAVs remain fixed over a short scheduling window. Under this quasi-static assumption, the wireless topology, link qualities, and available communication capacities among UAVs are determined by their current locations and channel conditions. Accordingly, the system must decide, for each task, which UAV serves as the access point, which UAVs participate in collaborative inference, and how intermediate features are transmitted across the UAV network, with the goal of minimizing end-to-end inference latency. Fig. 1 illustrates the quasi-static UAV collaborative inference scenario considered in this paper.

A. Quasi-Static UAV Network

Consider a UAV edge network consisting of a set of UAVs denoted by $\mathcal{U} = \{1, 2, \dots, U\}$. Each UAV $u \in \mathcal{U}$ is equipped with limited computing power, memory, and wireless communication resources. The position of UAV u is denoted by L_u . During each scheduling window, L_u is treated as a known parameter and is not subject to optimization. This assumption is reasonable for scenarios such as disaster response, where UAVs typically hover or move slowly. Within such a window, the system focuses on organizing inference resources based on the current network topology rather than controlling UAV trajectories.

Let C_u denote the available computing power of UAV u and M_u denote its available memory capacity for storing model parameters or modules. Both C_u and M_u are heterogeneous across UAVs due to differences in hardware platforms, battery status, and background workloads.

B. Wireless Model

The UAV network comprises both User-UAV access links and UAV-UAV links. Let $\gamma_{u,v}$ denote the signal-to-noise ratio (SNR) of the link from UAV u to UAV v . The link is considered available if $\gamma_{u,v}$ exceeds a predefined threshold γ_{th} . Accordingly, we define the link availability indicator as

$$g_{u,v} = \begin{cases} 1, & \text{if } \gamma_{u,v} \geq \gamma_{th}, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

In this work, intermediate features can be transmitted directly between UAVs only if $g_{u,v} = 1$. This design choice is motivated by three considerations. First, in quasi-static scenarios, UAV distances and channel conditions remain relatively stable within a scheduling window, making single-hop connectivity sufficient for most low-latency requirements. Second, multi-hop routing would couple the expert deployment problem with routing decisions, significantly increasing complexity. Third, intermediate features are typically large, so the cumulative delay and relay overhead of multi-hop transmission may outweigh its connectivity benefits. Consequently, multi-hop routing is left as a future extension, and we restrict all transmissions to single-hop reachable UAVs.

The channel gains are modeled using large-scale path loss. For the air-to-ground link, $h_{k,u} = \beta_0 d_{k,u}^{-\alpha_{acc}}$, where β_0 is the reference channel power gain at 1 m, $d_{k,u}$ is the distance, and α_{acc} is the path loss exponent. For the UAV-UAV link, $h_{u,v} = \beta_0 d_{u,v}^{-\alpha_{U2U}}$, with $d_{u,v}$ denoting the distance between UAVs u and v , and α_{U2U} the corresponding path loss exponent. The SNR of the UAV-UAV link from u to v is given by

$$\gamma_{u,v} = \frac{P_u h_{u,v}}{\sigma_{u,v}^2}, \quad (2)$$

where P_u is the transmit power of UAV u , $h_{u,v}$ is the channel gain, and $\sigma_{u,v}^2 = N_0 B_{u,v}$ is the noise power with N_0 denoting the noise power spectral density and $B_{u,v}$ the link bandwidth. The achievable data rate follows the Shannon capacity

$$R_{u,v} = B_{u,v} \log_2(1 + \gamma_{u,v}). \quad (3)$$

Similar to (2) and (3), the SNR of the air-to-ground access link from the ground node of task k to UAV u is $\gamma_{k,u} = \frac{P_k h_{k,u}}{\sigma_{k,u}^2}$, with noise power $\sigma_{k,u}^2 = N_0 B_{k,u}$. The capacity is $R_{k,u} = B_{k,u} \log_2(1 + \gamma_{k,u})$.

C. Collaborative Inference Task Model

Let $\mathcal{K}$ denote the set of inference tasks arriving within the current scheduling window. Each task $k \in \mathcal{K}$ is generated by a ground user or sensor, with input data size S_k^{in} and output result size S_k^{out} . The task first uploads its input data to an access UAV, after which one or more UAVs collaboratively execute the inference, and the final result is returned to users.

To maintain generality, we abstract the inference model as a set of computation units $\mathcal{B}$. Each unit $b \in \mathcal{B}$ represents a deployable module (e.g., an expert weight, a shard, or a layer block) with computation requirement F_b and memory footprint W_b . For task k , the required computation units form an ordered subset $\mathcal{B}_k \subseteq \mathcal{B}$. The computation units required by task k

form an ordered subset $\mathcal{B}_k \subseteq \mathcal{B}$ arranged according to their linear execution order, where $L_k = |\mathcal{B}_k|$ denotes the number of inference steps. In the AeroMoE design, these abstract computation units are instantiated as the layer-wise expert feed-forward network (FFN) modules of the MoE model.

The system determines the access UAV for each task, the set of participating UAVs, the assignment of computation units to UAVs, and the transmission paths for intermediate features. We introduce the following binary decision variables, $y_{k,u} \in \{0, 1\}$ indicates whether task k accesses UAV u and $s_{k,b,u} \in \{0, 1\}$ indicates whether computation unit b of task k is executed on UAV u . We introduce the binary variable $p_{k,l,u,v} \in \{0, 1\}$ to indicate whether intermediate features must be transmitted from u to v between the l -th and $(l+1)$ -th execution steps of task k . Both $s_{k,b,u}$ and $p_{k,l,u,v}$ are abstract decision variables defined for general collaborative inference.

D. End-to-End Delay Model

The end-to-end latency of task k consists of four components, i.e., uplink delay, computation delay, transmission delay, and downlink delay,

$$D_k = D_k^{\text{ul}} + D_k^{\text{comp}} + D_k^{\text{trans}} + D_k^{\text{dl}}. \quad (4)$$

They are given as

$$D_k^{\text{ul}} = \sum_{u \in \mathcal{U}} y_{k,u} \frac{S_k^{\text{in}}}{R_{k,u}}, \quad (5)$$

$$D_k^{\text{comp}} = \sum_{b \in \mathcal{B}} \sum_{u \in \mathcal{U}} s_{k,b,u} \frac{F_b}{C_u}, \quad (6)$$

$$D_k^{\text{trans}} = \sum_{l=1}^{L_k-1} \sum_{u \in \mathcal{U}} \sum_{v \in \mathcal{U}, v \neq u} p_{k,l,u,v} \frac{S_{k,l}^{\text{mid}}}{R_{u,v}}, \quad (7)$$

where $p_{k,l,u,v}$ can take the value 1 only if $g_{u,v} = 1$. If UAVs u and v are not directly connected, the intermediate features cannot be transmitted between them. In this case, the scheduler must select another reachable UAV for execution.

$$D_k^{\text{dl}} = \frac{S_k^{\text{out}}}{R_{u_{k,L_k}^*, k}}, \quad (8)$$

where L_k is the number of execution steps and $u_{k,l}^*$ denotes the UAV chosen to execute the l -th step of task k , so u_{k,L_k}^* is the UAV selected for the final step. The term $R_{u_{k,L_k}^*, k}$ represents the effective downlink rate from this UAV to the destination of task k .

E. Optimization Problem

Given UAV positions, link states, computation and memory resources, and the set of tasks, the objective is to minimize the expected end-to-end latency, which is

$$\min_{y, s, p} \mathbb{E}[D_{\text{total}}], \quad (9)$$

where $D_{\text{total}} = \sum_{k \in \mathcal{K}} \omega_k D_k$ and $\omega_k > 0$ is the priority weight of task k . Each task accesses exactly one UAV, i.e.,

$$\text{C1: } \sum_{u \in \mathcal{U}} y_{k,u} = 1, \quad \forall k \in \mathcal{K}. \quad (10)$$

Each computation unit must be assigned to exactly one UAV for execution, i.e.,

$$\text{C2: } \sum_{u \in \mathcal{U}} s_{k,b,u} = 1, \quad \forall k \in \mathcal{K}, b \in \mathcal{B}. \quad (11)$$

The total computation load on each UAV cannot exceed its available budget, i.e.,

$$\text{C3: } \sum_{k \in \mathcal{K}} \sum_{b \in \mathcal{B}} s_{k,b,u} F_b \leq C_u^{\text{max}}, \quad \forall u \in \mathcal{U}. \quad (12)$$

The memory usage on each UAV cannot exceed its capacity, i.e.,

$$\text{C4: } \sum_{b \in \mathcal{B}} m_{b,u} W_b \leq M_u, \quad \forall u \in \mathcal{U}, \quad (13)$$

and execution is permitted only on UAVs where the module is deployed, i.e.,

$$\text{C5: } s_{k,b,u} \leq m_{b,u}, \quad \forall k \in \mathcal{K}, b \in \mathcal{B}, u \in \mathcal{U}. \quad (14)$$

Intermediate feature transmission is restricted to single-hop links, i.e.,

$$\text{C6: } p_{k,l,u,v} = 1 \Rightarrow g_{u,v} = 1, \\ \forall k \in \mathcal{K}, l = 1, \dots, L_k - 1, u, v \in \mathcal{U}. \quad (15)$$

For each task k with linear execution order $b_{k,1} \prec_k b_{k,2} \prec_k \dots \prec_k b_{k,L_k}$, where $b_{k,l} \in \mathcal{B}$ and $L_k = |\mathcal{B}_k|$, if two consecutive modules are assigned to different UAVs, intermediate features must be transmitted between them,

$$\text{C7: } s_{k,b_{k,l},u} = 1 \wedge s_{k,b_{k,l+1},v} = 1 \wedge u \neq v \Rightarrow p_{k,l,u,v} = 1, \\ \forall k \in \mathcal{K}, l = 1, \dots, L_k - 1, u, v \in \mathcal{U}. \quad (16)$$

This formulation captures the essential characteristics of collaborative inference in quasi-static UAV edge networks, where task access, computation assignment, and feature transmission must be jointly optimized under resource and connectivity constraints. Subsequent sections will instantiate this general model for MoE inference by specializing the computation units to experts and incorporating similarity-aware deployment and scheduling.

IV. PROPOSED DESIGN

In the optimization problem (9), the system must simultaneously optimize the expert deployment variables $m_{e,u}$, task access variables $y_{k,u}$, expert execution variables $s_{k,l,u}$, and intermediate feature transmission variables $p_{k,l,u,v}$. The search space of this problem grows exponentially with the number of UAVs, experts, and tasks, rendering direct solution intractable under real-time constraints.

To address this challenge, AeroMoE decouples the joint optimization into five sequential stages as illustrated in Fig. 2. Each stage solves a smaller, tractable subproblem, and the stages are executed in a feed-forward manner, with the output of one stage serving as the input to the next. This design trades the global search capability of iterative alternating optimization for significantly lower computational complexity, which is essential for real-time operation within quasi-static UAV scheduling windows. The five stages are defined as follows.

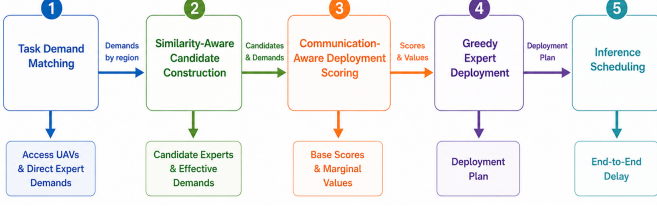


Fig. 2. AeroMoE decouples the joint optimization into five sequential stages, i.e., task demand matching, similarity-aware candidate construction, communication-aware dynamic deployment, and inference scheduling.

1) *Task Demand Matching*: Ground tasks are assigned to UAV regions according to minimum access delay. The direct expert demand distribution $\text{Demand}(r, u)$ is then computed for each UAV region.

2) *Similarity-Aware Candidate Construction*: For each originally required expert r , a substitutable expert set $\mathcal{A}(r)$ is constructed based on expert weight similarity $\text{Sim}(r, e)$. The original demands are then extended to effective demands $\text{Demand}^{\text{eff}}(e, u)$ over the candidate experts.

3) *Communication-Aware Expert Placement Scoring*: For each candidate deployment pair (e, u) , a base deployment score $\text{BaseScore}(e, u)$ is computed by jointly considering link availability $g_{v,u}$, UAV computing power C_u , and network topology.

4) *Dynamic Expert Placement*: A dynamic greedy algorithm then generates the final deployment decisions $\{m_{e,u}\}$ under per-UAV memory constraints.

5) *Inference Scheduling*: A per-layer greedy scheduler is designed to determine the end-to-end task latency D_{total} .

In this design phase, the abstract computation units $b \in \mathcal{B}$ from the System Model are instantiated as concrete MoE experts $e \in \mathcal{E}_{\text{cand}}$. The linear execution order $\prec_k$ is specialized to the layer-wise expert FFN sequence of the MoE model, where the expert required at step l is denoted by $e_{k,l}$. Accordingly, the abstract execution variables $s_{k,b,u}$ are refined to per-step execution indicators $s_{k,l,u}$, defined as

$$s_{k,l,u} = 1 \iff \text{the } l\text{-th expert of } k \text{ is executed on } u, \quad (17)$$

for all $k \in \mathcal{K}$, $l = 1, \dots, L_k$, and $u \in \mathcal{U}$. The transmission variables $p_{k,l,u,v}$ are no longer treated as explicit decision variables during deployment. Instead, they are implicitly determined during inference scheduling by the single-hop transmission from the current UAV cur to the target UAV u .

A. Task Demand Matching

This section defines the Task-UAV assignment and direct expert demand statistics.

1) *Default Task-UAV Assignment*: For each task $k \in \mathcal{K}$, AeroMoE assigns it to a default access UAV according to the minimum access delay, which is given by

$$u_k^a = \arg \min_{u \in \mathcal{U}} \frac{S_k^{\text{in}}}{R_{k,u}}. \quad (18)$$

After the assignment, all expert demands for task k are aggregated into the region of its default access UAV u_k^a for subsequent demand statistics.

2) *Direct Expert Demand*: Given the Task-UAV assignment, the demand for an original expert r in the region of UAV u is defined as

$$\text{Demand}(r, u) = \sum_{k: u_k^a = u} \omega_k q_{k,r}, \quad (19)$$

where ω_k is the weight of task k and $q_{k,r}$ denotes the demand intensity of expert r in task k . If the MoE model provides gating scores, $q_{k,r}$ is the sum of gating scores over all input tokens of task k for expert r ; otherwise, it is the number of times expert r appears in the expert sequence of task k . Note that $\text{Demand}(r, u)$ only accounts for the *originally required* experts. Similarity-based substitution and effective demand computation are handled in the next subsection.

B. Similarity-Aware Candidate Expert Construction

This section defines the original expert set, substitutable expert set, candidate expert set, and effective demand.

1) *Original Expert Set*: Let $\mathbf{e}_k = (e_{k,1}, e_{k,2}, \dots, e_{k,L_k})$ denote the original expert execution sequence of task k , where $e_{k,l}$ is the expert required at step l . The set of all originally required experts across tasks is

$$\mathcal{E}_{\text{req}} = \{r \mid \exists k \in \mathcal{K}, \exists l \in \{1, \dots, L_k\}, r = e_{k,l}\}. \quad (20)$$

2) *Substitutable Expert Set*: For any original expert $r \in \mathcal{E}_{\text{req}}$, its substitutable set consists of r itself and all sufficiently similar experts

$$\mathcal{A}(r) = \{r\} \cup \{e \in \mathcal{E} \mid \text{Sim}(r, e) \geq \xi\}. \quad (21)$$

The similarity between two experts is measured by the shifted cosine similarity of their flattened FFN weight matrices

$$\text{Sim}(r, e) = \frac{1}{2} \left(1 + \frac{\langle \text{vec}(\mathbf{W}_r), \text{vec}(\mathbf{W}_e) \rangle}{\|\text{vec}(\mathbf{W}_r)\|_2 \|\text{vec}(\mathbf{W}_e)\|_2} \right), \quad (22)$$

where $\mathbf{W}_r$ and $\mathbf{W}_e$ are the FFN weight parameters of experts r and e , respectively. Operator $\text{vec}(\cdot)$ stacks the columns of a matrix into a vector. This formulation ensures $\text{Sim}(r, e) \in [0, 1]$ and captures functional substitutability. The threshold ξ controls the size of $\mathcal{A}(r)$. A low ξ introduces many weakly related experts and inflates the search space, while a high ξ may exclude useful substitutes.

3) *Candidate Expert Set*: The candidate expert set is the union of all substitutable sets, which is given as

$$\mathcal{E}_{\text{cand}} = \bigcup_{r \in \mathcal{E}_{\text{req}}} \mathcal{A}(r). \quad (23)$$

This set contains both originally required experts and their similar substitutes. All subsequent deployment decisions are restricted to $\mathcal{E}_{\text{cand}}$.

4) *Effective Demand*: A candidate expert $e \in \mathcal{E}_{\text{cand}}$ can inherit part of the demand of the experts it can substitute. The effective demand of e in the region of UAV u is defined as

$$\text{Demand}^{\text{eff}}(e, u) = \sum_{r: e \in \mathcal{A}(r)} \text{Demand}(r, u) \theta(r, e), \quad (24)$$

where the discount factor $\theta(r, e)$ is

$$\theta(r, e) = \begin{cases} 1, & \text{if } e = r, \\ \text{Sim}(r, e), & \text{if } e \neq r. \end{cases} \quad (25)$$

Thus, an expert fully inherits the demand when it is the originally required expert and inherits a similarity-weighted fraction otherwise.

C. Communication-Aware Expert Placement Scoring

This section quantifies the benefit and cost of deploying expert e on UAV u . AeroMoE evaluates each deployment action (e, u) from four perspectives, i.e., communication-aware task demand benefit, computing power benefit, position benefit, and model size cost. These are combined into a base deployment score $\text{BaseScore}(e, u)$.

1) *Communication-Aware Task Demand Benefit*: This benefit of deploying expert e on UAV u is defined as

$$A(e, u) = \text{Demand}^{\text{eff}}(e, u) + \eta \sum_{v \neq u} \text{Demand}^{\text{eff}}(e, v) g_{v,u} \frac{R_{v,u}}{R_{\max}}, \quad (26)$$

where $\eta \in [0, 1]$ is a weighting coefficient that balances local and neighboring demand. The first term captures the local demand in the region of UAV u , reflecting the benefit of serving tasks that can directly access expert e without cross-UAV transmission. The second term accounts for the potential benefit of serving tasks in neighboring UAV regions via single-hop links. This term is active only when $g_{v,u} = 1$ and is weighted by the normalized link rate $R_{v,u}/R_{\max}$, where $R_{\max}$ is a reference value computed from all available UAV-UAV link rates at the beginning of the scheduling window.

Note that $A(e, u)$ does not explicitly incorporate intermediate feature size S^{mid} . As a static score computed during the deployment planning phase, it uses normalized link rate as a proxy for communication quality. The actual end-to-end transmission delay, which depends on the realized S^{mid} , is computed later during inference scheduling.

2) *Computing Power Benefit*: The computing power benefit of deploying expert e on UAV u is defined as

$$B(e, u) = \frac{C_u}{F_e}, \quad (27)$$

where C_u is the available computing power of UAV u and F_e is the computation requirement of expert e . This term favors assigning computation-intensive experts to UAVs with higher processing power.

3) *Position Benefit*: The position benefit of deploying expert e on UAV u is defined as

$$C(e, u) = \left(\sum_{v \in \mathcal{U}} \text{Demand}^{\text{eff}}(e, v) \right) \cdot \left(\sum_{v \neq u} g_{v,u} \right). \quad (28)$$

The first factor represents the total effective demand for expert e across all UAV regions, capturing its global importance. The second factor is the in-degree of UAV u in the single-hop communication graph, i.e., the number of UAVs that can reach u directly. Experts with high global demand are preferably placed on UAVs with high connectivity to minimize multi-hop feature transmissions.

4) *Base Deployment Score*: Combining the three benefit terms with a penalty yields the base deployment score, i.e.,

$$\text{BaseScore}(e, u) = \alpha A(e, u) + \beta B(e, u) + \gamma C(e, u) - \mu W_e, \quad (29)$$

where W_e is the memory footprint of expert e , and $\alpha, \beta, \gamma, \mu > 0$ are weighting coefficients. The term μW_e acts as a soft penalty that prefers smaller experts when other factors are comparable, thereby preserving memory headroom for subsequent deployments.

Since the four terms have different physical units, AeroMoE applies min-max normalization to each term before summation, mapping all values to the interval $[0, 1]$. The weights $\alpha, \beta, \gamma, \mu$ are determined via grid search in Section V.

D. Dynamic Greedy Expert Placement

This section presents a two-phase deployment strategy that guarantees coverage of originally required experts while controlling redundancy under per-UAV memory constraints.

In Phase 1, the algorithm ensures that every expert $r \in \mathcal{E}_{\text{req}}$ is covered by at least one deployed expert, which may be r itself or any sufficiently similar substitute from $\mathcal{A}(r)$. If no feasible candidate can cover certain original experts within the memory limit, the deployment is marked infeasible. This phase guarantees that every original expert request can be served during online inference.

In Phase 2, the algorithm iteratively selects additional experts while penalizing functional overlap. As deployment proceeds, the marginal benefit of placing an expert decreases if similar experts have already been deployed on the same UAV or its single-hop neighbors. To capture this effect, AeroMoE defines a redundancy neighborhood

$$\mathcal{A}_{\text{red}}(e) = \{e' \in \mathcal{E}_{\text{cand}} \setminus \{e\} \mid \text{Sim}(e, e') \geq \xi_{\text{red}}\}, \quad (30)$$

where $\xi_{\text{red}} = \xi$ by default. The redundancy cost of deploying expert e on UAV u at iteration t is

$$\begin{aligned} \text{Redundancy}_t(e, u) = & \sum_{\substack{e' \in \mathcal{A}_{\text{red}}(e) \\ m_{e',u}^{(t)} = 1}} \text{Sim}(e, e') \\ & + \eta_{\text{red}} \sum_{v \neq u} g_{v,u} \sum_{\substack{e' \in \mathcal{A}_{\text{red}}(e) \\ m_{e',v}^{(t)} = 1}} \text{Sim}(e, e'), \end{aligned} \quad (31)$$

where the first term penalizes local redundancy and the second term penalizes redundancy among single-hop neighbors.

The dynamic score at iteration t is obtained by subtracting the current redundancy cost from the static base score:

$$\text{Score}_t(e, u) = \text{BaseScore}(e, u) - \nu \text{Redundancy}_t(e, u). \quad (32)$$

To prevent bias toward memory-intensive experts, the algorithm further normalizes by memory footprint and selects candidates according to the unit-memory value

$$\text{Value}_t(e, u) = \frac{\text{Score}_t(e, u)}{W_e}. \quad (33)$$

Algorithm 1 summarizes the complete procedure. Phase 1 first guarantees coverage of all originally required experts.

Algorithm 1 Dynamic Expert Placement Algorithm

Require: Candidate expert set $\mathcal{E}_{\text{cand}}$; UAV set $\mathcal{U}$; base deployment score $\text{BaseScore}(e, u)$; expert memory footprint $\{W_e\}$; UAV memory capacity $\{M_u\}$; substitutable expert sets $\{\mathcal{A}(r)\}$; redundancy neighborhoods $\{\mathcal{A}_{\text{red}}(e)\}$; originally required expert set $\mathcal{E}_{\text{req}}$.

Ensure: Expert deployment decisions $\{m_{e,u}\}$.

- 1: Initialize $m_{e,u}$ and remaining memory $\tilde{M}_u$ for all $e \in \mathcal{E}_{\text{cand}}$, $u \in \mathcal{U}$; initialize copy count c_e for all $e \in \mathcal{E}_{\text{cand}}$.
- 2: *Phase 1: Coverage Pre-deployment*
- 3: **while** there exists an uncovered expert $r \in \mathcal{E}_{\text{req}}$ **do**
- 4: Construct the set of currently uncovered experts $\mathcal{R} \leftarrow \{r \in \mathcal{E}_{\text{req}} \mid \nexists (e, u) \text{ s.t. } m_{e,u} = 1 \text{ and } e \in \mathcal{A}(r)\}$.
- 5: **if** $\mathcal{R} = \emptyset$ **then**
- 6: **break**
- 7: **end if**
- 8: Build the set of feasible coverage candidates $\mathcal{P} \leftarrow \{(r, e, u) \mid r \in \mathcal{R}, e \in \mathcal{A}(r) \cap \mathcal{E}_{\text{cand}}, u \in \mathcal{U}, c_e < 1, W_e \leq \tilde{M}_u\}$.
- 9: **if** $\mathcal{P} = \emptyset$ **then**
- 10: Mark the current deployment as infeasible and **return** failure.
- 11: **end if**
- 12: Select the best coverage candidate $(r^*, e^*, u^*) \leftarrow \arg \max_{(r,e,u) \in \mathcal{P}} \frac{\text{BaseScore}(e,u)}{W_e}$.
- 13: Deploy the selected expert $m_{e^*,u^*} \leftarrow 1$, $\tilde{M}_{u^*} \leftarrow \tilde{M}_{u^*} - W_{e^*}$, $c_{e^*} \leftarrow c_{e^*} + 1$.
- 14: **end while**
- 15: *Phase 2: Dynamic Deployment*
- 16: $t \leftarrow 1$
- 17: **loop**
- 18: Update $\text{Redundancy}_t(e, u)$ for all e, u using (31).
- 19: Compute $\text{Value}_t(e, u)$ for all e, u using (33).
- 20: Construct the set $\mathcal{F}_t \leftarrow \{(e, u) \mid c_e < 1, W_e \leq \tilde{M}_u\}$.
- 21: **if** $\mathcal{F}_t = \emptyset$ **then**
- 22: **break**
- 23: **end if**
- 24: Select the best (e^*, u^*) , deploy the expert m_{e^*,u^*} .
- 25: $t \leftarrow t + 1$
- 26: **end loop**
- 27: **return** $\{m_{e,u}\}$

Phase 2 then iteratively selects the feasible candidate with the highest unit-memory value until no more candidates can be placed. The final deployment satisfies the memory capacity constraint by construction.

E. Lightweight Inference Scheduling

Once the expert deployment decisions $\{m_{e,u}\}$ are fixed, the online inference phase determines, for each arriving task, the UAV on which each expert should execute, whether a deployed similar expert can be used as a substitute, and how intermediate features should be routed. The flowchart of lightweight inference scheduling is shown as Fig. 3.

For each task k , inference begins at its default access UAV after the shared attention and gating layers are executed,

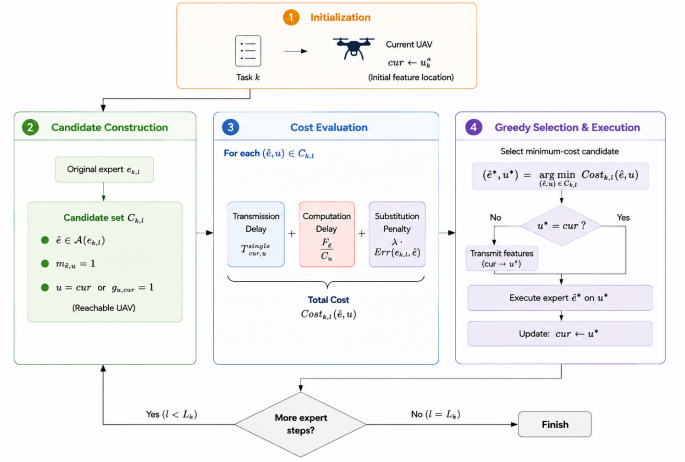


Fig. 3. Lightweight inference scheduling.

$\text{cur} \leftarrow u_k^a$, where cur tracks the UAV currently holding the intermediate features and is updated after each cross-UAV transmission.

For the l -th expert step of task k , let $e_{k,l}$ be the expert selected by the gating network. The candidate set of execution pairs $(\hat{e}, u)$ consists of all deployed experts $\hat{e} \in \mathcal{A}(e_{k,l})$ that are either local to the current or reachable UAV, and whose computation requirement does not exceed the remaining compute budget of the target UAV, i.e.,

$$\mathcal{C}_{k,l} = \{(\hat{e}, u) \mid \hat{e} \in \mathcal{A}(e_{k,l}), m_{\hat{e},u} = 1, u = \text{cur} \text{ or } g_{\text{cur},u} = 1, \tilde{C}_u \geq F_{\hat{e}}\}. \quad (34)$$

Here, $\hat{e}$ may be the originally required expert $e_{k,l}$ or any sufficiently similar deployed substitute, and u is the candidate execution UAV. The cost of executing expert $\hat{e}$ on UAV u at step l is defined as

$$\text{Cost}_{k,l}(\hat{e}, u) = T_{\text{cur},u}^{\text{single}}(S_{k,l-1}^{\text{mid}}) + \frac{F_{\hat{e}}}{C_u} + \lambda \cdot \text{Err}(e_{k,l}, \hat{e}), \quad (35)$$

where $T_{\text{cur},u}^{\text{single}}(S_{k,l-1}^{\text{mid}})$ is the single-hop transmission delay from the current UAV to UAV u (zero if $u = \text{cur}$); $F_{\hat{e}}/C_u$ is the computation delay on UAV u ; $\text{Err}(e_{k,l}, \hat{e}) = 1 - \text{Sim}(e_{k,l}, \hat{e})$ is the approximation error incurred by using a substitute (zero when $\hat{e} = e_{k,l}$); $\lambda > 0$ is a penalty weight that trades off execution latency against substitution error.

At each step, the scheduler greedily selects the minimum-cost candidate:

$$(\hat{e}^*, u^*) = \arg \min_{(\hat{e}, u) \in \mathcal{C}_{k,l}} \text{Cost}_{k,l}(\hat{e}, u). \quad (36)$$

If $u^* \neq \text{cur}$, the intermediate features are transmitted from cur to u^* over a single-hop link. The expert $\hat{e}^*$ is then executed on u^* , after which the current location is updated as $\text{cur} \leftarrow u^*$.

This per-step greedy selection, together with the candidate construction rules, ensures that each expert layer is executed on exactly one UAV while satisfying the single-hop transmission and per-UAV compute capacity constraints.

V. SIMULATION RESULTS

A. Simulation Setup

The simulation considers a quasi-static UAV edge network consisting of multiple UAVs. Each UAV is characterized by its three-dimensional position, computation capability C_u , memory capacity M_u available for expert parameters, and transmit power. When two UAVs satisfy the connectivity condition, the intermediate feature transmission is permitted; otherwise, direct intermediate feature transmission between them is not permitted in the current scheduling step. To emulate spatially heterogeneous task demands typical in UAV edge scenarios, we adopt a controlled random hotspot setup. The simulation area is $240 \text{ m} \times 240 \text{ m}$, with 4 UAVs deployed near the centers of 4 service regions. Their horizontal positions are perturbed with small random offsets, and their altitudes are uniformly sampled between 70 m and 90 m. The computing power and memory capacities of the UAVs are also randomly sampled to reflect resource heterogeneity. Tasks are generated as localized hotspots around each UAV, with distinct expert demand distributions across regions. This setup mimics realistic applications such as disaster monitoring and low-altitude inspection, where different geographic areas generate different types of inference requests.

The expert pool contains eight experts. Some experts have large memory footprints and high computing requirements, representing the primary experts originally required by tasks. Others are constructed as lightweight substitutes whose weight vectors are similar to those of the primary experts. This configuration creates realistic memory pressure, preventing all original experts from being replicated on every UAV, while also providing a verifiable structure for evaluating the similarity-based substitution mechanism. All random variables are controlled by fixed random seeds, and experimental results are averaged over multiple seeds to reduce the impact of particular random realizations.

Each task is defined by its ground location, input size S_k^{in} , output size S_k^{out} , per-layer intermediate feature sizes $S_{k,l}^{\text{mid}}$, weight ω_k , and expert execution sequence $\mathbf{e}_k$ determined by the gating network. Each expert is characterized by its memory footprint W_e , per-forward computation requirement F_e , and weight vector used for cosine similarity computation. When gating scores are unavailable, the demand intensity $q_{k,r}$ of expert r in task k is computed from its occurrence count in the expert sequence of task k .

The default algorithm parameters are set as follows: $\alpha = 0.4$, $\beta = 0.25$, $\gamma = 0.2$, $\mu = 0.15$, $\eta = 0.5$, $\nu = 0.3$, $\eta_{\text{red}} = 0.5$, $\lambda = 0.1$, and similarity threshold $\xi = 0.7$. For reproducibility, the default simulation and wireless channel parameters are summarized in Table I, the task and expert pool configurations are listed in Table II, and the algorithm hyperparameters along with their ranges used in sensitivity and scalability studies are provided in Table III.

B. Baselines and Metrics

To isolate the contribution of each design component in AeroMoE, we compare the following schemes.

TABLE I
SIMULATION SCENARIO AND WIRELESS SETTINGS.

Item	Setting
<i>Scenario scale</i>	
Scenario	Controlled random hotspot UAV scenario
Random seeds	$\{0, 1, 2, 3, 4\}$
Area	$240 \text{ m} \times 240 \text{ m}$
UAVs / tasks / experts	$U = 4, K = 40, E = 8$
Expert vector dimension	64
<i>UAV resources</i>	
Base positions	$(60, 60), (180, 60), (60, 180), (180, 180)$
Position perturbation	$x, y \sim \mathcal{U}[-10, 10] \text{ m}; H \sim \mathcal{U}[70, 90] \text{ m}$
Computing capability C_u	$\mathcal{U}[8, 11] \text{ GFLOP/s}$
Memory capacity M_u	$\mathcal{U}[850, 1050] \text{ MB}$
UAV transmit power P_u	$\mathcal{U}[10, 14] \text{ W}$
<i>Wireless channel</i>	
Reference gain β_0	1
Path-loss exponent	2.5
Noise density N_0	10^{-12} W/Hz
Bandwidth B	20 MHz
SNR threshold γ_{th}	5
Ground transmit power	5 W

TABLE II
TASK AND EXPERT SPECIFICATIONS.

Item	Setting
<i>Expert pool</i>	
Original experts E_0-E_3	$W_e \sim \mathcal{U}[620, 760] \text{ MB}, F_e \sim \mathcal{U}[1.0, 1.35] \text{ GFLOP}$
Substitute experts E_4-E_6	$W_e \sim \mathcal{U}[220, 320] \text{ MB}, F_e \sim \mathcal{U}[0.35, 0.52] \text{ GFLOP}$
Auxiliary expert E_7	$W_e \sim \mathcal{U}[280, 380] \text{ MB}, F_e \sim \mathcal{U}[0.50, 0.70] \text{ GFLOP}$
Similar pairs	$(E_0, E_4), (E_1, E_5), (E_2, E_6)$
Similarity noise scale	$\mathcal{U}[0.06, 0.10]$
<i>Task profile</i>	
Number of layers	$\mathcal{U}\{2, 3, 4\}$
Input size S_k^{in}	$\mathcal{U}[1.0, 2.2] \text{ Mbit}$
Output size S_k^{out}	$\mathcal{U}[10, 25] \text{ kbit}$
Mid-feature size $S_{k,l}^{\text{mid}}$	First value $\mathcal{U}[0.8, 1.4] \text{ Mbit}$, then multiplied by 0.9^l
Task priority ω_k	$\mathcal{U}[0.8, 1.4]$
Hotspot spread	Gaussian offset, $\sigma = 12 \text{ m}$
<i>Hotspot expert distributions</i>	
UAV 0	experts (0, 1, 3, 2), probabilities (0.75, 0.15, 0.07, 0.03)
UAV 1	experts (1, 3, 0, 2), probabilities (0.75, 0.15, 0.07, 0.03)
UAV 2	experts (2, 0, 3, 1), probabilities (0.75, 0.15, 0.07, 0.03)
UAV 3	experts (0, 2, 1, 3), probabilities (0.48, 0.48, 0.02, 0.02)

- **Proposed:** The complete AeroMoE framework, including similarity-aware candidate construction, effective demand extension, communication-aware scoring, coverage-aware pre-deployment, dynamic redundancy-penalized greedy placement, and per-layer scheduling.
- **Random Placement:** Candidate experts are randomly deployed subject to memory constraints, followed by the same per-layer single-hop scheduler used in the Proposed

TABLE III
ALGORITHM PARAMETERS AND SWEEP SETTINGS.

Item	Setting
<i>Default algorithm parameters</i>	
Similarity threshold	$\xi = 0.7$
Placement score weights	$\alpha = 0.4, \beta = 0.25, \gamma = 0.2, \mu = 0.15$
Remote-demand weight	$\eta = 0.5$
Redundancy weights	$\nu = 0.3, \eta_{\text{red}} = 0.5$
Scheduling penalty	$\lambda = 0.1$
Expert copy limit	One copy per expert in the default scenario
<i>Sensitivity studies</i>	
Similarity threshold	$\xi \in \{0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$
UAV memory scale	$\{0.9, 1.0, \dots, 1.4\}$
Mid-feature scale	$\{0.4, 0.6, \dots, 2.0\}$
<i>Scalability studies</i>	
UAV count	$U \in \{4, 6, \dots, 20\}; K = 60, E = 8$
Task count	$K \in \{20, 30, \dots, 100\}; U = 10, E = 8$
Expert count	$E \in \{8, 10, \dots, 24\}; U = 10, K = 80$

scheme.

- **Importance:** Experts are deployed in descending order of their global effective demand, without considering communication connectivity, UAV computation heterogeneity, or network position benefits.
- **No-Sim:** Similarity-based expert substitution is disabled; only originally required experts may be deployed and executed.

The primary performance metric is the total latency D_{total} . We further decompose the latency into access delay, expert computation delay, UAV-UAV transmission delay, and result return delay. To evaluate the effectiveness of similarity-based substitution, we also report the number of times substitute experts are used during inference. For robustness analysis, we conduct sensitivity studies with respect to the similarity threshold ξ , intermediate feature size, and UAV memory capacity. Finally, we evaluate the scalability of AeroMoE by varying the number of UAVs, the number of tasks, and the size of the expert pool.

C. Performance Analysis

1) *End-to-End Delay Performance:* Fig. 4 shows the average end-to-end latency of different methods across multiple random seeds. Compared with Random Placement, Importance, and No-Sim, AeroMoE reduces the average end-to-end latency by 24.1%, 21.1%, and 48.3%, respectively. The Proposed scheme achieves an average latency of 9878.9 ms, outperforming Random Placement (13014.9 ms) and Importance (12527.7 ms). No-Sim yields the highest average latency of 19107.5 ms, indicating that disabling similarity-based substitution forces the system to adopt suboptimal deployment and execution paths under tight memory constraints.

These results indicate that the performance gains of AeroMoE stem from the joint effect of demand-driven candidate construction, communication-aware placement scoring, and dynamic redundancy-penalized greedy selection, rather than from any single component. Random Placement lacks awareness of task demand and network topology, often placing

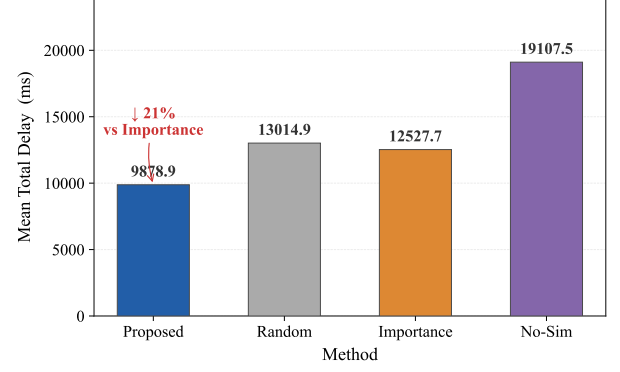


Fig. 4. Average end-to-end latency comparison of different expert deployment methods.

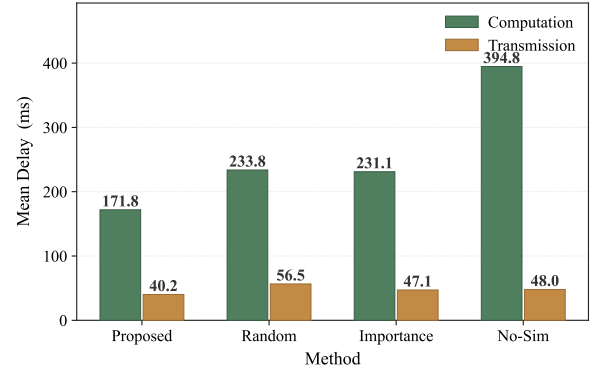


Fig. 5. End-to-end latency breakdown. AeroMoE reduces both computation and transmission overhead through communication-aware deployment and per-layer scheduling.

experts on UAVs with unfavorable communication or computation conditions. Importance considers only global demand and fails to account for the communication benefits of deploying experts on different UAVs. No-Sim, by disabling similarity-based substitution, loses the ability to expand the candidate set and reduce memory pressure through expert reuse.

2) *Delay Breakdown:* Fig. 5 presents the breakdown of end-to-end latency. The access and return delays are comparable across all methods, as they share the same task assignment policy and wireless channel model. The main differences lie in expert computation delay and UAV-UAV transmission delay. The Proposed scheme achieves an average computation delay of 171.8 ms, which is lower than Random Placement (233.8 ms), Importance (231.1 ms), and No-Sim (394.8 ms). Its average transmission delay is 40.2 ms, also lower than Random Placement (56.5 ms), Importance (47.1 ms), and No-Sim (48.0 ms).

These results are consistent with the design of the placement scoring mechanism in Section IV-C. The term $A(e, u)$ encourages the placement of experts in locations that can serve both local and single-hop neighboring demands, thereby reducing

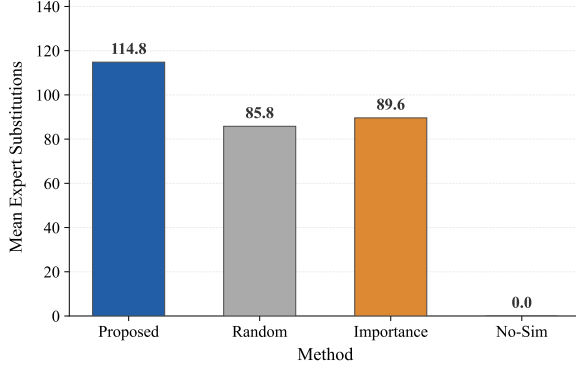


Fig. 6. Number of substitute expert usages. The Proposed scheme utilizes similar experts more effectively to cover originally required demands.

cross-UAV transmissions. The term $B(e, u)$ favors assigning computation-intensive experts to UAVs with higher processing capability. In addition, the dynamic redundancy penalty prevents the repeated deployment of functionally similar experts in the same region, allowing limited memory to be allocated to higher-gain candidates. No-Sim exhibits significantly higher computation delay because it cannot exploit similarity-based substitution and is forced to execute more expensive original experts. Although Random Placement and Importance can utilize substitute experts, they lack a comprehensive mechanism that jointly considers communication and computation conditions, resulting in higher computation and transmission overheads compared with the Proposed scheme.

3) *Similarity Substitution Analysis*: Fig. 6 shows the number of times substitute experts are used during inference. The Proposed scheme utilizes substitute experts 114.8 times on average, which is higher than Random Placement (85.8 times) and Importance (89.6 times). No-Sim, as expected, uses no substitutes. These results demonstrate that AeroMoE not only includes similar experts as deployment candidates but also actively leverages them during online scheduling to reduce both transmission and computation costs.

The benefits of similarity-based substitution arise at two stages. In the deployment stage, the substitutable set $\mathcal{A}(r)$ expands the candidate pool beyond originally required experts, while the effective demand $\text{Demand}^{\text{eff}}$ allows substitute experts to inherit part of the original demand. This provides greater deployment flexibility under memory constraints. In the scheduling stage, the error penalty term $\lambda(1 - \text{Sim})$ in the cost function prevents the selection of low-quality substitutes, maintaining a controlled trade-off between latency reduction and approximation error.

4) *Sensitivity Analysis*: Fig. 7 shows the impact of the similarity threshold ξ on system performance. A lower ξ enlarges the candidate set by including more substitute experts, thereby increasing deployment flexibility. As ξ increases, weakly similar substitutes are filtered out and the candidate space shrinks. The Proposed scheme consistently outperforms all baselines for $\xi \in [0.5, 0.9]$. This indicates that a moderate

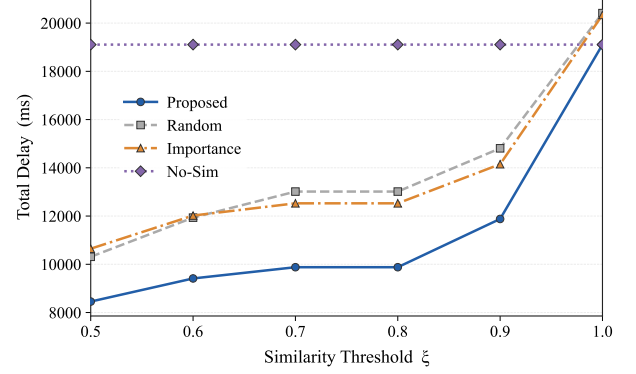


Fig. 7. Sensitivity analysis with respect to the similarity threshold ξ .

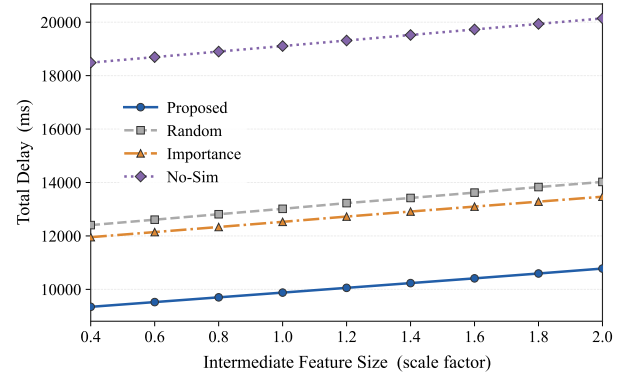


Fig. 8. Impact of intermediate feature size on end-to-end latency.

threshold expands the available options without allowing the excessive use of low-quality substitutes. The similarity requirement itself, combined with the approximation error penalty in the scheduling cost function, effectively prevents the selection of poor substitutes.

When $\xi = 1.0$, only identical experts are admitted into the candidate set, effectively disabling similarity-based substitution. In this case, the Proposed scheme degenerates into an exact-matching strategy, and its latency rises sharply to approach that of No-Sim. Random Placement and Importance also fail to achieve lower latency at this point, as their deployment strategies rely solely on randomness or global demand ordering and lack the coordinated mechanisms of coverage pre-deployment, communication-aware scoring, and dynamic redundancy control that enable more effective placement of exact experts.

Fig. 8 further examines the impact of intermediate feature size. As S^{mid} increases, the proportion of cross-UAV transmission delay in the end-to-end latency rises, causing all methods to experience higher latency. However, the Proposed scheme exhibits the smallest increase and maintains the lowest latency across all feature sizes. This robustness stems from two aspects of the design. First, the communication-aware placement

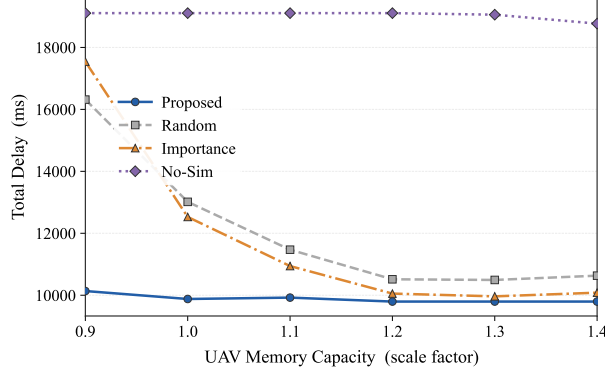


Fig. 9. Impact of UAV memory capacity on end-to-end latency and deployment scale.

scoring explicitly accounts for single-hop reachability and link rates, encouraging the deployment of high-demand experts on well-connected UAVs. Second, similarity-based substitution expands the set of feasible execution locations, reducing the need to transmit large intermediate features over low-rate links.

Fig. 9 shows the performance trends as UAV memory capacity varies. As memory becomes more abundant, all methods gain additional deployment space and achieve lower latency. Nevertheless, the Proposed scheme consistently maintains the lowest latency and exhibits the most stable performance. This advantage arises because AeroMoE uses coverage pre-deployment and similarity substitution to ensure that originally required experts can be served, while the dynamic greedy algorithm allocates remaining memory to high-gain placements. In contrast, Random Placement and Importance benefit more significantly from increased memory, indicating that they rely more heavily on extra capacity to compensate for suboptimal deployment decisions. No-Sim shows limited improvement with larger memory budgets, confirming that simply increasing capacity cannot fully substitute for the deployment flexibility provided by similarity-based candidate expansion.

5) *Scalability Analysis:* Beyond parameter sensitivity, we further evaluate the scalability of AeroMoE with respect to system size. Fig. 10 shows the results when increasing the number of UAVs while keeping the number of tasks and experts fixed. As the number of UAVs grows from 4 to 20, the average per-task latency of the Proposed scheme remains low and decreases slightly from approximately 175 ms to 162 ms, consistently outperforming all baselines. Although a larger number of UAVs provides more deployment options and shorter average access distances, these additional resources translate into stable latency gains only when the deployment strategy jointly exploits task demand, single-hop connectivity, and UAV computation heterogeneity. Random Placement and Importance suffer more from mismatched deployment locations when the number of UAVs is small, while No-Sim, despite benefiting from more UAVs, still exhibits significantly higher latency due to the lack of memory flexibility from similarity-based substitution. The Proposed curve shows only

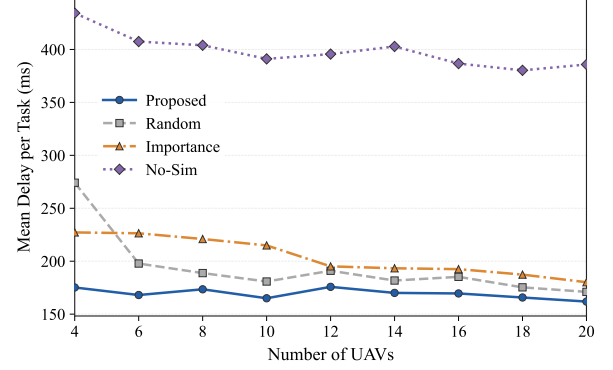


Fig. 10. Scalability with respect to the number of UAVs.

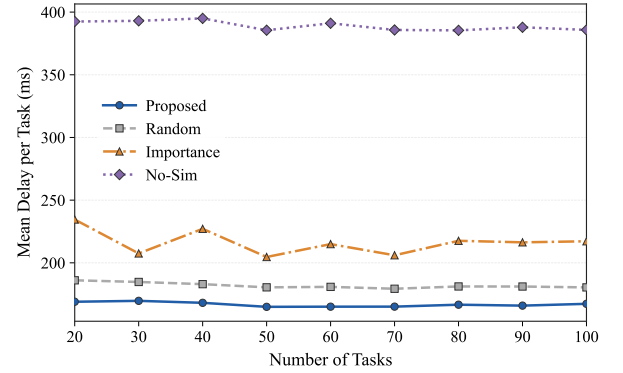


Fig. 11. Average per-task latency as the number of tasks increases.

minor fluctuations, indicating that topological randomness introduced by varying UAV counts does not undermine its performance advantage.

Fig. 11 presents the results when increasing the number of tasks from 20 to 100 while fixing the number of UAVs and experts. The average per-task latency of all methods remains relatively stable, suggesting that a larger task population primarily affects demand statistics and scheduling frequency rather than disrupting the single-hop execution structure. The Proposed scheme maintains the lowest latency across all task scales, demonstrating that demand-driven effective expert modeling can extract stable deployment preferences from a larger set of task samples. No-Sim consistently yields the highest latency, as the absence of substitute experts continues to amplify the overhead caused by limited coverage and constrained execution paths. Random Placement and Importance also exhibit higher latency than the Proposed scheme because their deployment strategies underutilize localized demand and link conditions.

Fig. 12 shows the results when increasing the expert pool size from 8 to 24 while fixing the number of UAVs and tasks. A larger expert pool expands the candidate deployment space but also intensifies memory competition. The Proposed scheme

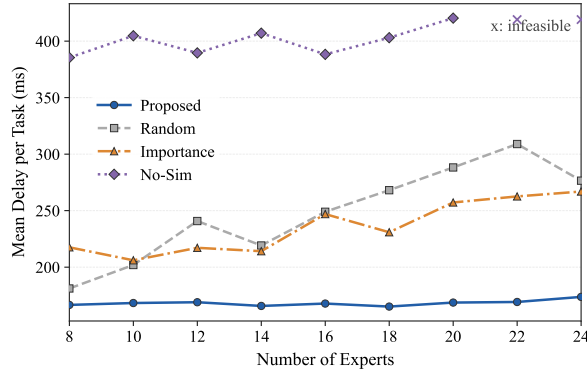


Fig. 12. Scalability with respect to expert pool size.

maintains the lowest latency and highest feasibility across the entire range, because similarity-based candidate construction converts newly added experts into usable substitutes, while the dynamic redundancy penalty prevents limited memory from being wasted on functionally overlapping placements. In contrast, Random Placement and Importance are more prone to low-gain deployments as the expert pool grows, leading to increased latency and larger fluctuations. No-Sim, which cannot exploit substitute experts, becomes increasingly constrained by original expert coverage requirements and even encounters infeasible cases near 22 and 24 experts. The Proposed curve rises only gradually with expert pool size, confirming that similarity-aware candidate construction and redundancy control effectively mitigate the memory contention caused by a larger expert set.

VI. CONCLUSION

This paper addressed the problem of collaborative MoE inference in UAV edge networks under stringent constraints on memory, computation, and communication. We proposed AeroMoE, a similarity-aware framework that integrates expert substitutability, task demand, UAV resource heterogeneity, and communication topology into a unified deployment and scheduling solution. Through extensive simulations in random hotspot UAV scenarios, AeroMoE achieved average end-to-end latency reductions of 24.1%, 21.1%, and 48.3% compared with random placement, importance-based placement, and similarity-agnostic deployment, respectively. These results demonstrate that jointly exploiting expert similarity for candidate expansion and communication-aware placement for topology-aware allocation can significantly improve inference efficiency in resource-constrained aerial edge environments. The proposed approach offers a practical pathway toward deploying large-scale intelligent models on memory-limited UAV platforms without relying on cloud connectivity.

REFERENCES

[1] P. McEnroe, S. Wang, and M. Liyanage, "A survey on the convergence of edge computing and AI for UAVs: Opportunities and challenges," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15 435–15 459, 2022.

[2] C. Chen, T. Zhang, W. Xu, X. Yang, and Y. Wang, "Multi-UAV cooperation based edge computing offloading in emergency communication networks," in *Proc. IEEE Wirel. Commun. Netw. Conf. (WCNC)*, 2023, pp. 1–6.

[3] Y. Guo, Z. Cheng, X. Tang, Z. Tu, and T. Lin, "Dynamic mixture of experts: An auto-tuning approach for efficient transformer models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 79 643–79 672.

[4] J. Shao and J. Zhang, "Communication-computation trade-off in resource-constrained edge inference," *IEEE Commun. Mag.*, vol. 58, no. 12, pp. 20–26, 2020.

[5] A. Sacco, F. Esposito, G. Marchetto, and P. Montuschi, "A self-learning strategy for task offloading in UAV networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4301–4311, 2022.

[6] Q. Li, *et al.*, "Memory-Constrained semantic segmentation for ultra-high resolution UAV imagery," *IEEE Robot. Autom. Lett.*, vol. 9, no. 2, pp. 1708–1715, 2024.

[7] X. Zhang, X. Wang, X. Xu, and Y. Zhao, "Demand learning and cooperative deployment of UAV networks," *Chin. J. Electron.*, vol. 31, no. 3, pp. 408–415, 2022.

[8] Y. Qu, *et al.*, "Elastic collaborative edge intelligence for UAV swarm: Architecture, challenges, and opportunities," *IEEE Commun. Mag.*, vol. 62, no. 1, pp. 62–68, 2024.

[9] M. A. Dhuheir, E. Baccour, A. Erbad, S. S. Al-Obaidi, and M. Hamdi, "Deep reinforcement learning for trajectory path planning and distributed inference in resource-constrained UAV swarms," *IEEE Internet Things J.*, vol. 10, no. 9, pp. 8185–8201, 2023.

[10] Z. Zhang, X. Liu, H. Cheng, C. Xu, and J. Gao, "Diversifying the expert knowledge for task-agnostic pruning in sparse Mixture-of-Experts," in *Proc. Findings Assoc. Comput. Linguist.:ACL 2025*, July 2025, pp. 86–102.

[11] S. He, D. Dong, L. Ding, and A. Li, "Towards efficient mixture of experts: A holistic study of compression techniques," [Online]. Available: <https://arxiv.org/abs/2406.02500>, 2024.

[12] L. Li, *et al.*, "Sub-MoE: Efficient mixture-of-expert LLMs compression via subspace expert merging," in *Proc. AAAI Conf. Artif. Intell.*, 2026.

[13] R. Kong, Y. Li, W. Wang, L. Kong, and Y. Liu, "Serving MoE models on resource-constrained edge devices via dynamic expert swapping," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2799–2811, 2025.

[14] H. Zhao, Z. Qiu, H. Wu, Z. Wang, Z. He, and J. Fu, "HyperMoE: Towards better mixture of experts via transferring among experts," in *Proc. Annu. Meet. Assoc. Comput. Linguist. (Volume 1: Long Papers)*, Aug. 2024, pp. 10 605–10 618.

[15] Q. Yan, Y. Li, N. Ma, and F. Wan, "Research on the structure and realization of Mixture-of-Experts," in *Proc. Int. Conf. Comput. Eng. Appl. (ICCEA)*, 2024, pp. 354–359.

[16] D. Dai, *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in Mixture-of-Experts language models," in *Proc. Annu. Meet. Assoc. Comput. Linguist.*, Aug. 2024, pp. 1280–1297.

[17] J. Shao, Y. Mao, and J. Zhang, "Learning task-oriented communication for edge inference: An information bottleneck approach," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 197–211, 2022.

[18] X. Xia, S. M. M. Fattah, and M. A. Babar, "A survey on UAV-enabled edge computing: Resource management perspective," *ACM Comput. Surv.*, vol. 56, no. 3, Oct. 2023.

[19] J. Yan, W. Wang, J. Liu, J. Deng, H. Yuan, and Y. Zhu, "Task demand-oriented collaborative offloading and deployment strategy in software-defined UAV-assisted edge networks," *IEEE Sens. J.*, vol. 25, no. 1, pp. 1641–1655, 2025.

[20] F. Chen, P. Li, S. Pan, L. Zhong, and J. Deng, "Giant could be tiny: Efficient inference of giant models on resource-constrained UAVs," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21 170–21 179, 2024.

[21] Z. Liang, X. Lyu, C. Ren, N. Li, and K. Li, "Communication-Efficient topology orchestration for distributed learning in UAV networks," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, 2024, pp. 662–667.

[22] W. Zhuang, Y. Mao, I. W.-H. Ho, and X. Yu, "UAV-Assisted cooperative edge inference for Low-Altitude economy via MoE-based hierarchical deep reinforcement learning," [Online]. Available: <https://arxiv.org/abs/2605.19290>, 2026.

[23] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, 2019.

[24] J. Zhu, Z. Han, and H. V. Poor, "Multi-UAV networks for search and rescue with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4910–4920, 2019.

- [25] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [26] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, 2020.
- [27] F. Jia, *et al.*, "CoDL: Efficient CPU-GPU Co-execution for deep learning inference on mobile devices," in *Proc. Annu. Int. Conf. Mobile Syst., Appl., Serv. (MobiSys)*, 2022, pp. 209–221.
- [28] J. S. Jeong, *et al.*, "BAND: Coordinated multi-DNN inference on heterogeneous mobile processors," in *Proc. Annu. Int. Conf. Mobile Syst., Appl., Serv. (MobiSys)*, 2022, pp. 235–247.
- [29] M. Wang, S. Ding, T. Cao, Y. Liu, and F. Xu, "ASYMO: Scalable and efficient deep-learning inference on asymmetric mobile CPUs," in *Proc. Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2021, pp. 215–228.
- [30] N. Shazeer, *et al.*, "Outrageously large neural networks: The Sparsely-Gated Mixture-of-Experts layer," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [31] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res. (JMLR)*, vol. 23, no. 120, pp. 1–39, 2022.
- [32] D. Lepikhin, *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," [Online]. Available: <https://arxiv.org/abs/2006.16668>, 2021.
- [33] A. Q. Jiang, *et al.*, "Mixtral of Experts," [Online]. Available: <https://arxiv.org/abs/2401.04088>, 2024.
- [34] Y. Zhou, *et al.*, "Mixture-of-Experts with expert choice routing," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022.
- [35] N. Du, *et al.*, "GLaM: Efficient scaling of language models with Mixture-of-Experts," [Online]. Available: <https://arxiv.org/abs/2112.06905>, 2022.
- [36] R. Y. Aminabadi, *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2022, pp. 646–660.
- [37] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "ORCA: A distributed serving system for Transformer-Based generative models," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2022, pp. 521–538.
- [38] Z. Li, *et al.*, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2023, pp. 663–679.
- [39] G. Zhu, *et al.*, "SMoE: An algorithm-system Co-Design for pushing MoE to the edge via expert substitution," [Online]. Available: <https://arxiv.org/abs/2508.18983>, 2025.